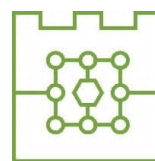




**Politechnika Krakowska  
im. Tadeusza Kościuszki**  
Wydział Informatyki i Telekomunikacji



**Michał Bąk**

Numer albumu: 144753

**Porównanie metod uczenia przez wzmacnianie w Ultimate Texas Hold'em z niepełną informacją: wpływ reprezentacji stanu i funkcji nagrody na jakość strategii**

**Comparison of reinforcement learning methods in Ultimate Texas Hold'em under imperfect information: the impact of state representation and reward function on strategy quality**

**Praca dyplomowa magisterska  
na kierunku Informatyka**

Praca wykonana pod kierunkiem:  
**mgr inż. Piotr Biskup**

**Kraków, 2026**

# SPIS TREŚCI

|                                                            |          |
|------------------------------------------------------------|----------|
| <b>1 Agenci bazowi i infrastruktura eksperymentalna</b>    | <b>3</b> |
| 1.1 Wspólny interfejs agenta . . . . .                     | 3        |
| 1.2 Prowadzenie pojedynczego epizodu . . . . .             | 3        |
| 1.3 Agent losowy . . . . .                                 | 4        |
| 1.4 Agent regułowy . . . . .                               | 4        |
| 1.4.1 Reguły decyzyjne . . . . .                           | 4        |
| 1.4.2 Implementacja strategii regułowej . . . . .          | 5        |
| 1.5 Symulacja wielu epizodów i jej powtarzalność . . . . . | 6        |
| 1.6 Metryki oceny . . . . .                                | 6        |
| 1.7 Porównanie agentów bazowych . . . . .                  | 7        |
| 1.8 Zapis wyników eksperymentu . . . . .                   | 8        |
| 1.9 Walidacja infrastruktury eksperymentalnej . . . . .    | 8        |
| 1.10 Podsumowanie . . . . .                                | 8        |

## 1. Agenci bazowi i infrastruktura eksperymentalna

W tym rozdziale przedstawiam projekt agentów bazowych oraz infrastruktury służącej do uruchamiania i oceny strategii w środowisku Ultimate Texas Hold'em. Rolę agentów bazowych jako punktów odniesienia dla metod uczenia przez wzmacnianie opisałem w rozdziale ??.

Zaprojektowałem i zaimplementowałem wspólny kontrakt agenta, mechanizm pojedynczego epizodu i symulację wielu rozdań, agenta losowego oraz agenta regułowego. Infrastrukturę uzupełniłem o mechanizm zbierania metryk i zapis wyników eksperymentów. Wszystkie te elementy korzystają z publicznego interfejsu środowiska, o którym pisałem w rozdziale ??.

### 1.1. Wspólny interfejs agenta

Zdefiniowałem wspólny kontrakt `Agent`, który określa, w jaki sposób agent komunikuje się ze środowiskiem. Kontrakt ten udostępnia metodę `select_action()`, która przyjmuje obiekt `UTHObservation` i zwraca podjętą przez agenta decyzję typu `Action`.

Wykorzystanie protokołu do konstrukcji kontraktu pozwala mi zdefiniować agenta w sposób elastyczny: klasa agenta nie musi dziedziczyć po wspólnej klasie bazowej, lecz musi udostępniać metodę o wymaganej sygnaturze. Dzięki temu ten sam mechanizm prowadzenia rozgrywki mogą wykorzystać dla agenta losowego, regułowego i uczącego się bez znajomości ich wewnętrznej implementacji.

Przypomnę, że agent otrzymuje obserwację zawierającą informacje dostępne graczowi. Pełny obiekt `RoundState` nie jest częścią interfejsu, dlatego implementacja strategii nie uzyskuje dostępu do kart krupiera, kart spalonych ani kolejności pozostałych kart w talii.

### 1.2. Prowadzenie pojedynczego epizodu

W tym celu zaimplementowałem osobny moduł, który nazywam `runnerem`. Zdefiniowałem w nim funkcję `play_round()`, która łączy agenta z silnikiem `UTHGame`. Funkcja ta rozpoczyna rozdanie, przekazuje bieżącą obserwację agentowi, wykonuje wybraną akcję i powtarza ten proces do osiągnięcia stanu terminalnego.

Każda decyzja zaakceptowana przez środowisko zostaje zapisana w kolejności wykonania. Po zakończeniu rozdania funkcja zwraca obiekt `EpisodeResult`, który zawiera sekwencję akcji, wynik rozdania oraz pełne rozliczenie zakładów Ante, Blind i Play.

Model `EpisodeResult` udostępnia również wynik netto, łączną wartość postawionych zakładów, liczbę decyzji i mnożnik zakładu Play. Stanowi w ten sposób lekką reprezentację zakończonego epizodu.

### 1.3. Agent losowy

Najprostszą strategię bazową stanowi agent losowy. Nie analizuje on wartości kart ani prawdopodobieństwa wygranej, a dla każdej obserwacji wybiera jedną z legalnych akcji z jednakowym prawdopodobieństwem: jeżeli w danej fazie legalne są na przykład trzy akcje, każda z nich ma szansę wyboru równą jednej trzeciej. W obserwacji `UTHObservation` agent dostaje zbiór legalnych akcji do wykonania. Akcje te są posortowane zgodnie z kolejnością wartości enuma `Action`, przez co wynik generatora pseudolosowego nie zależy od kolejności iterowania po zbiorze.

Środowisko i agent posiadają niezależne generatory pseudolosowe. Generator środowiska odpowiada za kolejność kart, natomiast generator agenta odpowiada wyłącznie za wybór akcji. Rozdzielając źródła losowości, uzyskałem możliwość zmiany strategii agenta, na przykład jego seedu albo całej implementacji, bez zmiany przebiegu rozdań. Dzięki temu, jak pokazuję w sekcji 1.7, mogę porównywać agenta losowego i agenta regułowego na dokładnie tym samym harmonogramie talii.

Napisałem testy obejmujące zgodność ze wspólnym protokołem, wybieranie wyłącznie legalnych akcji, powtarzalność decyzji dla tego samego seedu oraz obsługę niepoprawnych danych wejściowych. Jest również test potwierdzający, że agent nie podejmuje decyzji dla obserwacji terminalnej. Testy samego runnera opisuję w sekcji 1.9.

### 1.4. Agent regułowy

Agent losowy jest podstawowym punktem odniesienia, lecz nie korzysta w ogóle z wiedzy o grze. Dlatego w celu uzyskania lepszego punktu odniesienia zdefiniowałem agenta regułowego, który w przeciwieństwie do agenta losowego podejmuje decyzje na podstawie kart widocznych w obserwacji oraz aktualnej fazy rozdania. Przyjęte reguły oparłem na uproszczonej strategii *Ultimate Texas Hold'em* opublikowanej przez *Wizard of Odds* [1].

#### 1.4.1. Reguły decyzyjne

Zanim zostaną odkryte karty wspólne, agent podejmuje decyzję o zakładzie *Play* w wysokości czterokrotności *Ante* dla układów początkowych przedstawionych w tabeli 1.1.

**Tabela 1.1.** Układy początkowe uprawniające do zakładu *Play* 4× przed flopem

| Układ początkowy | Warunek                                      |
|------------------|----------------------------------------------|
| Para             | od trójek wzwyż                              |
| As               | z dowolną kartą towarzyszącą                 |
| Król             | z dowolną kartą w tym samym kolorze          |
| Król             | z kartą pięć lub wyższą w różnych kolorach   |
| Dama             | z kartą sześć lub wyższą w tym samym kolorze |
| Dama             | z kartą osiem lub wyższą w różnych kolorach  |
| Walec            | z kartą osiem lub wyższą w tym samym kolorze |
| Walec            | z dziesiątką w różnych kolorach              |

Dla pary dwójek oraz wszystkich pozostałych układów agent wykonuje akcję CHECK. Ta strategia nie wykorzystuje zakładu BET\_3X.

Jeżeli agent nie wykonał zakładu przed flopem, po odkryciu trzech kart wspólnych wybiera BET\_2X, gdy posiada dwie pary lub silniejszy układ, ukrytą parę z wyjątkiem pary dwójek albo cztery karty do koloru z własną kartą tego koloru o randze co najmniej dziesięć.

Przez ukrytą parę rozumiem parę wykorzystującą co najmniej jedną z dwóch prywatnych kart agenta. Jeżeli żaden z tych warunków nie jest spełniony, agent wykonuje akcję CHECK.

W ostatnim punkcie decyzyjnym agent wykonuje zakład BET\_1X, jeżeli posiada ukrytą parę lub silniejszy układ. Dla słabszych układów wykorzystywana jest reguła oparta na liczbie kart mogących poprawić układ krupiera (ang. *outs*) do układu pokonującego gracza. Jeżeli liczba takich kart jest mniejsza niż 21, agent wykonuje BET\_1X. W przeciwnym przypadku wybiera FOLD [1].

Jest to strategia deterministyczna: dla tej samej obserwacji agent zawsze zwraca tę samą akcję. Nie jest to jednak strategia optymalna w sensie teorii gier, ponieważ opiera się na uproszczonych heurystykach pochodzących z publicznie dostępnej strategii Wizard of Odds, a nie na pełnym przeszukaniu przestrzeni decyzji. W szczególności reguła stosowana na riverze przybliża liczbę outów krupiera, czego dokładność opisuję w podrozdziale 1.4.2. Mimo tych ograniczeń agent regułowy stanowi silniejszy punkt odniesienia niż agent losowy.

#### 1.4.2. Implementacja strategii regułowej

W celu implementacji agenta regułowego zdefiniowałem klasę RuleBasedAgent, która implementuje interfejs agenta pokerowego. Oddzieliłem reguły decyzyjne od klasy agenta, co pozwala mi na ich łatwe modyfikowanie w razie potrzeby.

Funkcje `should_raise_preflop()`, `should_raise_flop()` oraz `should_raise_river()` odpowiadają za ocenę obserwacji w poszczególnych fazach rozdania.

Do podjęcia decyzji na riverze zaimplementowałem funkcję `count_dealer_outs()`. Tworzy ona zbiór kart niewidocznych dla gracza, a następnie każdą z nich analizuje jako pojedynczą potencjalną kartę prywatną krupiera. Jeżeli dołączenie danej karty do kart wspólnych prowadzi do układu silniejszego od najlepszego układu agenta, karta zostaje zaklasyfikowana jako out.

Takie podejście to odzwierciedlenie uproszczonej reguły 21 outów. Nie jest ono pełnym przeszukaniem wszystkich możliwych dwukartowych układów krupiera. Uwzględnia pojedyncze niewidoczne karty, które same prowadzą do układu pokonującego gracza.

### 1.5. Symulacja wielu epizodów i jej powtarzalność

Aby sprawnie przeprowadzać eksperymenty, składające się z wielu rozdań, zdefiniowałem obiekty `SimulationConfig` i `SimulationResult`.

Konfiguracja symulacji przechowuje uporządkowaną sekwencję seedów wykorzystywanych do tworzenia talii. Liczba seedów określa jednocześnie liczbę rozgrywanych epizodów.

Funkcja `run_simulation()` dla każdego seedu tworzy osobną talię oraz nową instancję środowiska `UTHGame`. Następnie wykorzystuje wspomnianą funkcję `play_round()` do rozegrania pełnego epizodu. Wyniki wszystkich rozdań są przechowywane w obiekcie `SimulationResult`.

W taki sposób oddzieliłem konfigurację eksperymentu od implementacji konkretnego agenta. Ten sam obiekt `SimulationConfig` może zostać wykorzystany do oceny wielu różnych strategii.

Przy projektowaniu infrastruktury eksperymentalnej przyświecał mi cel łatwego odtwarzania przeprowadzonych symulacji. Dla każdego epizodu zapisuję seed talii, dzięki temu ponowne utworzenie talii z tym samym seedem daje tę samą kolejność kart. Było to dla mnie istotne, ponieważ kiedy porównuję agentów między sobą, mam pewność, że różnice w wynikach wynikają z ich strategii, a nie z losowego układu kart.

### 1.6. Metryki oceny

Aby móc porównywać skuteczność agentów między sobą, potrzebuję metryk umożliwiających ocenę jakości strategii. Zdefiniowane tu miary nie są specyficzne dla agentów bazowych, dlatego w niezmiennionej postaci wykorzystuję je również do oceny agentów uczących się w kolejnych rozdziałach pracy. Za podstawową miarę jakości strategii uznałem wynik netto wyrażony w jednostkach zakładu Ante. I tak dla symulacji składającej się z  $N$  epizodów empiryczną wartość oczekiwaną oszacowałem jako średni wynik netto na jedno rozdanie:

$$\widehat{EV} = \frac{1}{N} \sum_{i=1}^N r_i, \quad (1.1)$$

gdzie  $r_i$  oznacza wynik netto  $i$ -tego epizodu wyrażony w jednostkach Ante.

Oprócz średniego wyniku obliczam również całkowity wynik netto oraz średnią i całkowitą wartość postawionych zakładów. Zmienność wyników można oszacować za pomocą odchylenia standardowego próby:

$$s = \sqrt{\frac{\sum_{i=1}^N (r_i - \bar{r})^2}{N - 1}}. \quad (1.2)$$

Na podstawie odchylenia standardowego próby wyznaczam błąd standardowy średniej:

$$SE = \frac{s}{\sqrt{N}}. \quad (1.3)$$

Użyte tu oznaczenia ( $\widehat{EV}$  dla estymatora wartości oczekiwanej,  $s$  dla odchylenia standardowego próby oraz  $SE$  dla błędu standardowego średniej) są standardową notacją statystyczną.

Poza tymi miarami, do oceny jakości strategii dołączam również końcowe wyniki rozdań oraz częstotliwość wykonywania poszczególnych akcji.

### 1.7. Porównanie agentów bazowych

W celu zweryfikowania procesu eksperymentalnego, porównałem agenta losowego i agenta regułowego. Każdy z tych agentów rozegrał 10 000 rozdań. Seed generatora budującego harmonogram seedów talii wynosił 20260815, natomiast generator decyzji agenta losowego zainicjalizowano seedem 20260816. Sam obiekt UTHGame nie otrzymuje własnego seedu w tym eksperymencie, powtarzalność zapewnia wyłącznie jawnie przekazana talia `Deck(seed=...)`. Uzyskane podstawowe metryki przedstawiam w tabeli 1.2.

**Tabela 1.2.** Wyniki agentów bazowych dla 10 000 rozdań

| Agent          | $\widehat{EV}$ | $s$<br>(odch. std.) | $SE$<br>(błąd std.) | Śr. stawka | Wynik netto |
|----------------|----------------|---------------------|---------------------|------------|-------------|
| RandomAgent    | -0,46435       | 4,46422             | 0,04464             | 4,7395     | -4643,5     |
| RuleBasedAgent | 0,03865        | 4,08130             | 0,04081             | 4,1905     | 386,5       |

Z zebranych wyników wynika, że agent losowy uzyskał średni wynik  $-0,46435$  jednostki Ante na rozdanie, a agent regułowy osiągnął w tej samej próbie średni wynik  $0,03865$ , czyli wynik wyższy o około  $0,503$  jednostki Ante na rozdanie. Jednocześnie średnia wartość środków stawianych przez agenta regułowego była niższa.

Pomimo tego, że agent regułowy uzyskał dodatni wynik w tej konkretnej próbie, nie interpretuję tego jako dowodu dodatniej rzeczywistej wartości oczekiwanej strategii. Błąd standardowy jego średniego wyniku wyniósł około  $0,04081$ , czyli wartość zbliżoną do samego oszacowania  $\widehat{EV}$ . Ten wynik eksperymentu służy jedynie jako punkt odniesienia dla późniejszej oceny agentów uczących się.

W celu zobrazowania wyników końcowych agentów bazowych przedstawiam w tabeli 1.3 rozkład wyników rozdań.

**Tabela 1.3.** Rozkład wyników agentów bazowych dla 10 000 rozdań

| Agent          | Wygrana | Wygrana krupiera | Fold   | Remis |
|----------------|---------|------------------|--------|-------|
| RandomAgent    | 45,01%  | 42,75%           | 8,49%  | 3,75% |
| RuleBasedAgent | 47,92%  | 31,75%           | 16,79% | 3,54% |

Agent regułowy częściej kończył rozdania foldem, lecz znacznie rzadziej doprowadzał do rozstrzygnięcia zakończonego wygraną krupiera.

### 1.8. Zapis wyników eksperymentu

Zdecydowałem się na zapis wyników eksperymentu w dwóch formatach: JSON i CSV. Plik JSON zawiera konfigurację eksperymentu oraz zagregowane metryki obu agentów. Plik CSV przechowuje dane poszczególnych epizodów, w tym indeks rozdania, seed talii, nazwę agenta, wynik netto, całkowitą stawkę, wynik końcowy i sekwencję wykonanych akcji.

Taki podział na dwa osobne pliki pozwala mi zestawiać wyniki obu agentów dla tych samych seedów talii oraz dosyć łatwo tworzyć wizualizacje.

### 1.9. Walidacja infrastruktury eksperymentalnej

Pokryłem testami zarówno scenariusze pojedynczych epizodów, jak i symulacje wielu rozdań. Na poziomie pojedynczego epizodu testami runnera objąłem wszystkie legalne ścieżki zakończenia rozdania. Na poziomie symulacji sprawdziłem zgodność liczby epizodów z konfiguracją, powtarzalność wyników dla tych samych seedów oraz wykorzystanie tych samych talii przez porównywanych agentów.

Testami agregacji metryk weryfikuję obliczanie wyniku całkowitego, średniego wyniku, wartości postawionych zakładów, odchylenia standardowego, błędu standardowego oraz liczników wyników i akcji. Osobnymi testami pokryłem zapis wyników eksperymentu.

### 1.10. Podsumowanie

Podsumowując ten rozdział, zdefiniowałem dwa punkty odniesienia dla dalszych eksperymentów. `RandomAgent` reprezentuje strategię pozbawioną wiedzy o wartości kart, natomiast `RuleBasedAgent` wykorzystuje deterministyczny zestaw reguł.

Przygotowałem infrastrukturę, która umożliwia prowadzenie powtarzalnych symulacji, agregowanie wyników i zapisywanie danych poszczególnych epizodów. Ta infrastruktura stanowi fundament do porównywania agentów wykorzystujących metody uczenia przez wzmacnianie z ustalonymi strategiami bazowymi w kolejnych etapach pracy.



# **Spis rysunków**

# Spis tabel

|     |                                                                      |   |
|-----|----------------------------------------------------------------------|---|
| 1.1 | Układy początkowe uprawniające do zakładu Play 4× przed flopem . . . | 4 |
| 1.2 | Wyniki agentów bazowych dla 10 000 rozdań . . . . .                  | 7 |
| 1.3 | Rozkład wyników agentów bazowych dla 10 000 rozdań . . . . .         | 7 |

## **Bibliografia**

- [1] W. of Odds, *Ultimate Texas Hold'em*, <https://wizardofodds.com/games/ultimate-texas-hold-em/>, Dostęp: 2026-07-07.